

Ejercicios Tema 6. Documentación

Objetivos

Aplicar los conceptos de documentación del software mediante actividades de análisis, corrección, generación de JavaDoc y evaluación crítica de documentación realizada manualmente y con ayuda de IA.

Ejercicio 1. Detección de errores en JavaDoc

Lee el siguiente código e identifica **al menos 8 errores** relacionados con la documentación JavaDoc. Después, reescribe correctamente los comentarios.

```
/**
 * Clase Solo dice "Clase", tendria q explicarse mejor
 * @author Germán Gascón <ggascon@gmail.com>
 */
public class Calculadora {

    /**
     * Calcula la suma de dos números enteros.
     *
     * @param a Primer número a sumar.
     * @param b Segundo número a sumar.
     * @return El resultado de la suma de a y b.
     */
    public int sumar(int a, int b) {
        return a + b;
    }

    /**
     * Realiza la división entera de dos números.
     *
     * @param a Dividendo de la operación.
     * @param b Divisor de la operación.
     * @return El cociente de la división entera.
     * @throws ArithmeticException Si el divisor (b) es igual a cero.
     */
    public int dividir(int a, int b) {
        if (b == 0) {
            throw new ArithmeticException("División por cero");
        }
        return a / b;
    }
}
```

Ejercicio 2. Documentación de una clase

Documenta completamente la siguiente clase utilizando JavaDoc.

```
//creamos la clase llamada temporizador
public class Temporizador {
    /**
     * Cantidad de segundos almacenados.
     */
    private int segundos;
    /**
     * Constructor del temporizador.
     *
     * @param segundosIniciales Valor inicial en segundos.
     */
    public Temporizador(int segundosIniciales) {
        this.segundos = segundosIniciales;
    }

    /**
     * Incrementa el tiempo del temporizador.
     *
     * @param segundosExtra Segundos que se añadirán.
     * @throws IllegalArgumentException
     * si el valor es negativo.
     */
    public void incrementar(int segundosExtra) {
        if (segundosExtra < 0) {
            throw new IllegalArgumentException("No se permiten valores negativos");
        }
        segundos += segundosExtra;
    }
    /**
     * Reinicia el temporizador a cero.
     */
    public void reiniciar() {
        segundos = 0;
    }

    /**
     * Devuelve los segundos actuales.
     *
     * @return Cantidad de segundos.
     */
    public int getSegundos() {
        return segundos;
    }
}
```

Ejercicio 3. Selección de etiquetas adecuadas

Indica qué etiquetas JavaDoc deberían utilizarse para cada uno de los siguientes casos. No es necesario escribir toda la documentación completa; basta con indicar las etiquetas adecuadas y justificar brevemente la respuesta.

a) Método que recibe dos parámetros y devuelve un valor.

Etiquetas: @param y @return.

@param se utiliza para describir cada dato que recibe el método como entrada.

@return sirve para indicar el valor que devuelve el método después de ejecutarse.

b) Método que puede lanzar `IllegalArgumentException`.

Etiqueta: @throws.

Esta etiqueta se usa para explicar qué excepción puede producirse durante la ejecución y cuál sería la causa.

c) Clase pública de una biblioteca que empezó a existir en la versión 2.0.

Etiqueta: @since.

Permite indicar desde qué versión del proyecto existe una clase, método o funcionalidad.

d) Método obsoleto que debe sustituirse por otro.

Etiqueta: @deprecated.

Se utiliza cuando un método ya no es recomendable y existe otra alternativa más actual para reemplazarlo.

e) Método cuya documentación necesita enlazar con otro método relacionado.

Etiqueta: @see.

Sirve para añadir referencias a otras clases, métodos o documentación relacionada que pueda ser útil consultar.

f) Clase cuyo autor y versión deben mostrarse en la documentación.

Etiquetas: @author y @version.

@author identifica a la persona que desarrolló la clase y @version indica la versión actual del archivo o componente.

Ejercicio 4. Evaluación de la documentación de un proyecto real

Analiza la documentación del proyecto **jsoup**, una biblioteca Java muy conocida para trabajar con HTML y XML. El repositorio oficial en GitHub dispone de un README descriptivo, y el proyecto ofrece además una documentación API/Javadoc pública.

Material a consultar

- Repositorio: [jhy/jsoup](https://github.com/jhy/jsoup) en GitHub.
- Documentación [API/Javadoc oficial](#) de jsoup.

Enunciado

Evalúa la calidad de la documentación del proyecto respondiendo a las siguientes cuestiones:

- ¿El README explica claramente qué es el proyecto y para qué sirve?
Sí. Explica desde el inicio que jsoup es una librería Java para trabajar con HTML y XML, indicando sus funciones principales.
- ¿Incluye instrucciones de uso o ejemplos?
Sí. Incluye ejemplos básicos de código y explica cómo añadir la dependencia con Maven o Gradle.
- ¿La documentación API es fácil de navegar?
Sí. Usa el formato Javadoc estándar y las clases están bien organizadas.
- ¿Los nombres de clases y métodos resultan claros?
Sí. Los nombres son intuitivos y fáciles de entender.
- ¿La documentación está orientada a desarrolladores?
Sí. Está enfocada en explicar cómo usar la librería de forma práctica.
- ¿Qué aspectos mejorarías del README?
Añadiría una sección de errores frecuentes y más ejemplos rápidos.
- ¿Qué aspectos mejorarías del Javadoc o documentación API?

Incluiría ejemplos cortos en métodos complejos y una guía de cambios entre versiones.

Ejercicio 5. Crear y documentar una aplicación

Implementa en Java una aplicación que simule partidos de fútbol simplificados por consola.

La aplicación deberá contar con un objeto **Narrador** encargado de describir cada jugada.

Estructura

Jugador

Cada jugador tendrá:

- dorsal
- nombre y apellidos
- portero (0-100)
- defensa (0-100)

- regate (0-100)
- pase (0-100)
- tiro (0-100)

Equipo

Cada equipo tendrá:

- nombre
- array de 11 jugadores
- cantidad de goles marcados en el partido

Pelota

La pelota tendrá:

- referencia al equipo en posesión
- índice del jugador que la posee (0..10)

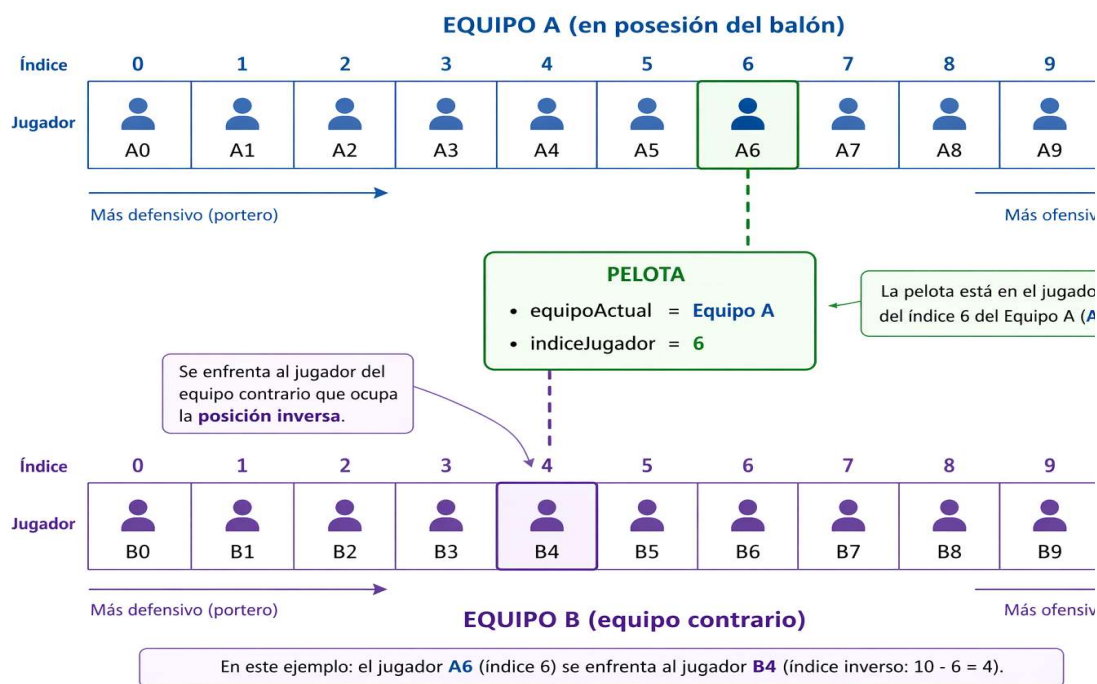
Desarrollo del Juego

El objetivo es que la pelota vaya avanzando posiciones en el array hasta llegar a una posición de tiro para meter gol.

El jugador con la pelota se enfrenta al rival en posición inversa:

- índice 10 con índice 0
- 9 con 1
- 8 con 2
- etc.

Excepto en el caso del tiro, donde el jugador rival siempre será el portero (ver siguiente apartado).



Acciones posibles

En cada turno, el jugador con la pelota elegirá aleatoriamente una acción válida que puede ser:

- **Pase**

Se comparan las propiedades:

- **pase** del atacante
- **defensa** del rival

Si tiene éxito, el balón pasa al siguiente compañero del array, es decir, la pelota avanza una posición en el array.

Restricción: el jugador con índice 10 (último), no puede realizar pases, solo tiro.

- **Regate**

Se compara:

- **regate** del atacante
- **defensa** del rival

Si tiene éxito, el jugador avanza una posición (se intercambia con el compañero siguiente en el array) y la pelota avanza una posición, de esta forma continua apuntando al mismo jugador pero que ahora está una posición más adelantada.

Restricción: el jugador con índice 10 (último), no puede realizar regates, solo tiro.

- **Tiro**

Solo se pueden jugadores cuyo índice en el array sea ≥ 7 .

Se compara:

- **tiro** ajustado por distancia/índice. Por cada posición alejada del índice 10, el

tiro se reduce un 10%. El cálculo sería:

$\text{tiroCalculado} = \text{tiroBase} * (1 - (10 - \text{índice}) * 0.10)$

- **portero** rival (jugador 0)

Aleatoriedad

Todas las acciones se les aplicará una variación aleatoria entre **-10%** y **+10%** para que las jugadas no produzcan siempre el mismo resultado.

Partido

- Se generan 2 equipos con 11 jugadores aleatorios cada uno usando la librería **DataFaker** para generar los nombres y apellidos.
- Se sortea el saque inicial para decidir qué equipo tiene la pelota inicialmente.
- El partido dura 3 minutos reales. Ayúdate de:

`System.currentTimeMillis()`

tomando valores al principio y después en cada acción, para calcular el tiempo total transcurrido.

- Entre acciones habrá pausa de 2 segundos para que dé tiempo a seguir el partido:

`Thread.sleep(2000);`

Narrador

Debe narrar al menos:

- pases
- regates
- robos
- tiros
- paradas
- goles
- resultado final

Por ejemplo algunos mensajes del narrador podrían ser:

"Juan García combina con Asier González"

"Pedro Martínez intenta el pase pero Unai López le adivina la acción y recupera la pelota"

"Diego Moncho dispara y ... Pablo Rivero muy atento atrapa la pelota sin problemas"

Pasos

- Implementar las clases necesarias para definir la lógica y la funcionalidad

requerida del simulador.

- Documentar con JavaDoc: clases, atributos, constructores, métodos públicos y excepciones.
- Generar el JavaDoc final, para ello debes crear una tarea Gradle para configurar la generación de la documentación.